

齿槽力矩补偿

采用方法：

离线标定的“位置查表 + q 轴电流前馈”齿槽补偿

一、国内外研究分析

1.1 工业界最常用：位置 LUT

这个方法的核心是：让电机低速匀速转一圈或多圈，记录每个位置需要多少 q 轴电流来抵消齿槽力矩，生成一张补偿表；之后运行时根据当前位置查表，把补偿电流叠加到力矩指令上。

商业伺服驱动器文档里，Kollmorgen、Synapticon、Novanta Motion、Harmonic Drive/YukonDrive 都是类似思路。Kollmorgen AKD 文档明确说 cogging compensation 是通过额外 feedforward current 抵消 cogging，表格按电机位置存储补偿值，旋转轴一圈有 8192 个表项，并且驱动器可以自动检测 cogging 并生成补偿表。

<https://webhelp.kollmorgen.com/kas/Content/UsersManual/Cogging%20Compensation.htm>

Kollmorgen 另一份说明还写得更具体：补偿信号加到 q 轴电流指令/速度控制器输出，记录时让电机恒速运动，记录 q 轴电流，滤波、去掉直流分量后写入表格；旋转电机通常按一机械圈周期重复。

<https://www.kollmorgen.com/en-us/developer-network/cogging-suppression>

Synapticon 也是类似：先做一次记录，运行时根据单圈位置读取补偿值并加到 torque reference；它还要求编码器最低 1024 cpr，推荐 4096 cpr 以上，记录时电机不要被机械阻挡，最好无负载，而且建议先完成系统辨识和位置环自动整定。Synapticon™ | Servo Drives for Robotics & Automation

Novanta Motion 的文档也直接把 cogging compensation 描述成基于 electrical angle 选索引的 LUT，表值单位是安培，运行时作为 torque loop output 的 offset 注入。Cogging Compensation

Harmonic Drive/YukonDrive 的手册同样是 teach-in：记录 q-current，去 offset，取反后作为 feedforward，支持按电角度或绝对位置补偿，并用线性插值。harmonicdrive.de

优点：

- 工业界常见，工程实现简单。
- 很适合低速伺服，因为齿槽力矩在低速时最明显。
- 算法计算量极小，F2800157 完全够用。

- 不需要复杂电机有限元模型。
- 和你现有摩擦补偿可以自然叠加。
- 对非周期轨迹也有效，因为它只依赖当前位置，不依赖运动轨迹。

缺点：

- 必须有稳定、重复的角度零点。非绝对编码器需要每次 homing，否则查表相位会错。
- 标定时必须尽量无负载，否则会把重力、线缆拖拽、外部负载也学进去。
- 表格会受到电流采样、驱动板电流比例、FOC 角度标定影响。
- 如果电机、驱动器、电流采样比例变了，需要重新标定。
- 它只补偿“位置重复”的扰动，不能替代摩擦补偿和外部负载扰动观测。

小结：

匹配度最高，而且简单实现。现在已经有机械参数辨识和摩擦前馈，正好可以在辨识完成后进入“齿槽 teach-in”状态，用 `Iq_fbk * Kt` 或 `Iq_fbk` 记录补偿表。

1.2 谐波模型前馈补偿

这个方法不是存完整 LUT，而是把齿槽力矩写成傅里叶级数：

$$T_{cog}(\theta) = \sum_{k=1}^N a_k \cos(k\theta) + b_k \sin(k\theta)$$

运行时用少量正弦、余弦项生成补偿电流。学术上很多 torque ripple/cogging ripple 都会用谐波表达。KIT/Schaeffler 的谐波电流注入论文提到，转矩脉动可以分成 cogging torque 和 ripple torque，其中 cogging torque 来自转子磁通和定子磁阻角度变化的相互作用，并且角度相关量可以用 Fourier series 处理。



icit_Langheck_final.pdf
1.88MB



优点：

- 存储量小，不需要 1024/2048 点表。
- 输出平滑，不容易有查表量化噪声。
- 适合写论文，因为形式更“模型化”。
- 可以通过 FFT 看主导谐波，只补主要阶次。

缺点：

- 要先知道或辨识主导谐波阶次。
- 如果齿槽波形尖锐，需要很多谐波，最后还不如 LUT。
- 相位误差会明显影响补偿效果。
- 工业驱动器更常直接用表格，而不是只让用户手动填傅里叶参数。

小结：需要确定谐波阶次，对相位敏感，调试不如 LUT 直接，暂时不用

1.3 扰动观测器 DOB / 转矩观测器 + 前馈补偿

这种方法用电流、速度、惯量、阻尼等信息估计负载扰动力矩，再把估计出来的周期性扰动按位置整理成补偿。国内《永磁同步电机转矩脉动抑制方法》这类文章就是采用转矩观测器估计转矩脉动，再结合位置信号生成前馈转矩电流，补偿到电流指令端。MDPI 2019 的一篇论文也提出 position-based repetitive control observer，把模型参考扰动观测器和基于位置的重复控制结合，用于齿槽力矩估计和补偿。



appls-ci-10-00096.pdf
7.94MB



优点：

- 可以在线估计，不一定完全依赖离线测量。
- 可以结合 J_m 、 B_m 、 C_m ，因为已经有机械模型。
- 对慢变扰动、负载扰动有一定适应能力。
- 做学术创新比单纯 LUT 更好讲。

缺点：

- 依赖 J 、 B 、 K_t 、电流反馈、速度反馈 的准确性。
- 速度微分/低速噪声会影响观测结果。
- 观测器带宽太低会有相位滞后，太高会放大噪声。
- 如果没有位置周期内部模型，普通 DOB 很难精准补齿槽力矩。
- 工业落地复杂度高于 LUT。

小结：依赖参数准确性，低速噪声大，难分离齿槽/摩擦/负载，暂时不用

1.4 迭代学习控制 ILC / 位置重复控制 PBRC / AFC 自适应前馈消除

这类方法把齿槽力矩看成“随位置周期重复”的扰动，每转一圈更新一次补偿量。RSS 2014 的机器人论文提出 position-based characterization anticogging：用伺服系统本来就有的位置传感器，把齿槽

力矩映射到转子位置，再按位置回放补偿；文中实验在 11 个电机上平均去除了约 69% 的 torque ripple。



p42.pdf
4.14MB



2024 年中文论文中也有基于 AFC 的齿槽转矩补偿和编码器校准方法，实验声称采用 AFC 后 torque 和 speed fluctuations 降低 95%，并强调这是不改硬件的软件补偿方法。



j.mtmt.1005-2402.2024.08.028.pdf
1.71MB



优点：

- 可以在线学习，不一定需要一次性标定。
- 对周期性扰动效果好。
- 可以同时处理编码器周期误差、齿槽力矩等周期项。
- 论文价值高，容易讲“自适应”“学习”。

缺点：

- 参数调试复杂。
- 对重复运动或稳定转速条件比较依赖。
- 收敛过程可能影响运动安全。
- 工业调试成本高，不如 LUT 直观。
- 如果你的速度/位置反馈有噪声，学习表可能被污染。

小结：需要重复运动和收敛过程，容易学进其他误差，暂时不用

1.5 谐波电流注入 HCI / 电流波形优化

这个方法不是直接查表补齿槽，而是通过注入特定谐波电流，让电机产生相反的谐波转矩，抵消转矩脉动。Sensors 2020 的论文把 current harmonic injection 描述为转矩脉动抑制中常用技术之一，通常在旋转坐标系电流指令层面注入谐波，主要修改产生转矩的 i_q 。

同一篇论文也指出，基于 FEA 优化电流谐波后，两个测试 PMSM 的 torque ripple 分别降低了 16 倍左右和 13 倍左右，但这种方法需要较复杂的模型和电流波形设计。

优点：

- 对电磁转矩脉动、反电势谐波、电流谐波有效。
- 高速工况、牵引电机、复杂 PMSM 设计中有价值。

- 可结合 FEA 或电机磁链模型做优化。

缺点：

- 不只是齿槽力矩补偿，更多是综合 torque ripple suppression。
- 需要知道电机电磁模型、反电势谐波或 FEA 结果。
- 会增加电流谐波，可能增加铜耗、噪声和电流环压力。
- 对你目前的低速关节伺服项目来说，复杂度偏高。

小结：更偏电磁转矩脉动抑制，模型要求高，工程改动大，暂时不用

综上，采用方法1。方法1是最简单最基础的，也是工业界常用方法，后面方法2-5适合在实现方法1后再进行优化。

二、具体方法

可以在把齿槽力矩补偿作为力矩前馈里面的第三个前馈项：摩擦项管速度相关扰动，齿槽项管位置相关扰动。

目前工程状态：

已经有 FOC，可以得到 `Iq_ref / Iq_fbk / rotor angle / velocity`；

已经做过机械参数辨识，得到了 `Jm、Bm、Cm`；

已经在做摩擦补偿；

现在要补的是另一类扰动：**齿槽力矩**。

位置 LUT 齿槽力矩补偿

2.1 本质

齿槽力矩 `Tcog` 是电机永磁体和定子齿槽之间由于磁阻变化产生的周期性扰动力矩。它最典型的特点是：

$$T_{cog} = T_{cog}(\theta_m)$$

也就是说，它主要和**电机转子机械位置**有关。

工业伺服里常用的 anti-cogging 做法，就是建立一张表：

$$\theta_m \rightarrow I_{q,cog}$$

也就是：电机转到这个角度时，需要额外加多少 q 轴电流，才能抵消这个位置上的齿槽力矩。

可以理解为，让速度环不要再费力去“追着齿槽力矩打补丁”，而是在速度环动作之前，**提前把齿槽力矩对应的电流补上**。

$$T_e^* = T_{PI} + T_{fric} + T_{cog,ff}(\theta_m)$$

$$I_q^* = \frac{T_e^*}{K_t}$$

其中 K_t 是电机转矩常数。

2.2理论推导

2.2.1电机机械方程

在电机轴侧，可以写成：

$$J_m \dot{\omega}_m = T_e - T_f(\omega_m) - T_{cog}(\theta_m) - T_L$$

其中：

- J_m ：电机侧等效转动惯量；
- ω_m ：电机机械角速度；
- T_e ：电磁转矩；
- T_f ：摩擦转矩；
- T_{cog} ：齿槽力矩；
- T_L ：外部负载扰动。

FOC 中电磁转矩近似为：

$$T_e = K_t I_q$$

摩擦项已经辨识过，近似为：

$$T_f(\omega_m) = B_m\omega_m + C_m \operatorname{sgn}(\omega_m)$$

代入机械方程：

$$J_m\dot{\omega}_m = K_t I_q - B_m\omega_m - C_m \operatorname{sgn}(\omega_m) - T_{cog}(\theta_m) - T_L$$

移项得到：

$$K_t I_q = J_m\dot{\omega}_m + B_m\omega_m + C_m \operatorname{sgn}(\omega_m) + T_{cog}(\theta_m) + T_L$$

这就是齿槽补偿的理论基础。

2.2.2为什么低速匀速转动可以辨识齿槽力矩？

标定齿槽力矩时，我们让电机低速匀速转动。低速匀速时：

$$\dot{\omega}_m \approx 0$$

空载：

$$T_L \approx 0$$

那么上式简化为：

$$K_t I_q \approx B_m\omega_m + C_m \operatorname{sgn}(\omega_m) + T_{cog}(\theta_m)$$

所以：

$$T_{cog}(\theta_m) \approx K_t I_q - B_m\omega_m - C_m \operatorname{sgn}(\omega_m)$$

如果直接用电流形式：

$$I_{q,cog}(\theta_m) \approx I_q - \frac{B_m\omega_m + C_m \operatorname{sgn}(\omega_m)}{K_t}$$

意思是：

在低速匀速转动时测到的 q 轴电流，扣掉摩擦需要的电流，剩下的主要就是齿槽补偿电流。

2.2.3为什么最好正转和反转都采？

因为摩擦是跟速度方向有关的，而齿槽力矩主要跟位置有关。

正转时：

$$K_t I_{q,+}(\theta) = B_m \omega_0 + C_m + T_{cog}(\theta)$$

反转时：

$$K_t I_{q,-}(\theta) = -B_m \omega_0 - C_m + T_{cog}(\theta)$$

把两者相加再除以 2：

$$\frac{K_t I_{q,+}(\theta) + K_t I_{q,-}(\theta)}{2} = T_{cog}(\theta)$$

$$T_{cog}(\theta) \approx \frac{K_t}{2} [I_{q,+}(\theta) + I_{q,-}(\theta)]$$

电流形式就是：

$$I_{q,cog}(\theta) \approx \frac{I_{q,+}(\theta) + I_{q,-}(\theta)}{2}$$

这个公式的好处是：正反转平均可以天然抵消一部分粘性摩擦和库仑摩擦。

反转采样时，要把数据按照相同的机械位置 θ_m 对齐，而不是按照采样时间对齐。

2.2.4LUT 表建立

假设你把一圈电机机械角度分成 N 个点。

例如： $N=1024$

每个表项对应一个角度区间：

$$\Delta\theta = \frac{2\pi}{N}$$

第 k 个表项对应：

$$\theta_k = k \frac{2\pi}{N}$$

$k=0,1,2,\dots,N-1$

要建立的表就是：

$$\text{LUT}[k] = I_{q,\text{cog}}(\theta_k)$$

运行时，根据当前电机角度 θ_m ，先做一圈取模：

$$\theta_{\text{mod}} = \theta_m \bmod 2\pi$$

然后计算索引：

$$x = \frac{\theta_{\text{mod}}}{2\pi} N$$

$$k = \lfloor x \rfloor$$

$$\alpha = x - k$$

为了避免表格跳变：

$$I_{q,\text{cog}}(\theta_m) = (1 - \alpha)\text{LUT}[k] + \alpha\text{LUT}[(k + 1) \bmod N]$$

这样运行时补偿电流就是平滑的。

工程中使用角度：机械角度

2.2.5取均值

标定出来的原始表可能长这样：

$$I_{\text{raw}}(\theta) = I_{\text{dc}} + I_{\text{cog}}(\theta) + \text{noise}$$

其中 I_{dc} 是一个直流偏置，可能来自：

- 轻微负载；
- 电流采样偏置；
- 速度环积分输出；
- 标定速度下的平均摩擦；
- 电机安装角度导致的平均力矩。

但是齿槽力矩是一圈周期扰动，它的平均值理论上应该接近 0：

$$\frac{1}{2\pi} \int_0^{2\pi} T_{cog}(\theta) d\theta \approx 0$$

所以生成最终 LUT 前要去均值：

$$\bar{I} = \frac{1}{N} \sum_{k=0}^{N-1} I_{raw}[k]$$

$$I_{cog}[k] = I_{raw}[k] - \bar{I}$$

Kollmorgen 的资料中也提到，记录的 q 轴电流值需要滤波，并且要移除 constant component，因为只需要补偿电流纹波部分。

2.2.6 在线补偿公式

最终速度环里面使用：

$$I_q^* = I_{q,PI} + I_{q,fric} + I_{q,cog}(\theta_m)$$

$$I_q^* = I_{q,PI} + \frac{B_m \omega_m + C_m \operatorname{sgn}(\omega_m)}{K_t} + \operatorname{InterpLUT}(\theta_m)$$

限幅：

$$I_q^* = \operatorname{sat}(I_q^*, -I_{q,max}, I_{q,max})$$

齿槽补偿表的值会直接变成驱动器输出电流的一部分。Kollmorgen 文档也提醒，cogging compensation value 会直接转化为供应给驱动器的电流，所以不能让表里出现异常大的补偿值。

三、实验方案

实验目标

建立一张齿槽补偿表：

$$\text{LUT}[k] = I_{q,\text{cog}}(\theta_k)$$

然后验证补偿前后：

- 低速速度波动是否变小；
- q 轴电流周期波动是否变小；
- 位置跟踪误差是否变小；
- 手转或慢速运行时卡顿感是否减轻。

实验一：补偿前基线测试

目的

先确认你的电机确实有明显齿槽扰动，并记录补偿前的数据。

条件

建议先空载或轻载。

速度给定：5rpm

可以测试几组：1 2 5 10

Kollmorgen 的资料中提到 teach 时可以用低速，甚至推荐约 1 rpm 以获得更好的补偿表质量；其旧版说明中也提到旋转电机记录默认可用 5 rpm。

实验二：齿槽表正转采集

目的

采集正转时每个位置需要的 q 轴电流。

步骤：

1. 关闭齿槽补偿
2. 低速匀速正转

实验三：齿槽表反转采集

目的

用反转数据抵消摩擦影响。

实验四：正反转融合生成最终 LUT

直接取正反转平均

$$LUT_{raw}[k] = \frac{LUT_+[k] + LUT_-[k]}{2}$$

然后去均值：

$$\bar{I}_{raw} = \frac{1}{N} \sum_{k=0}^{N-1} LUT_{raw}[k]$$

$$LUT[k] = LUT_{raw}[k] - \bar{I}_{raw}$$

然后做平滑滤波。

简单一点可以做循环滑动平均：

$$LUT_f[k] = \frac{1}{2M+1} \sum_{i=-M}^M LUT[(k+i) \bmod N]$$

M=2-5

代码啥逻辑示意

(此代码为模板，不是最终效果，里面的变量名可能与原工程不一样，但是逻辑可以参考)

1.建表阶段

代码块

```
1 #define COG_LUT_SIZE 1024
```

```

2
3  float cog_sum_pos[COG_LUT_SIZE];
4  float cog_sum_neg[COG_LUT_SIZE];
5  uint16_t cog_cnt_pos[COG_LUT_SIZE];
6  uint16_t cog_cnt_neg[COG_LUT_SIZE];
7
8  float cog_lut[COG_LUT_SIZE];
9
10 uint16_t get_cog_index(float theta_m)
11 {
12     float theta = fmodf(theta_m, TWO_PI);
13     if (theta < 0.0f) theta += TWO_PI;
14
15     float x = theta * ((float)COG_LUT_SIZE / TWO_PI);
16     uint16_t k = (uint16_t)x;
17
18     if (k >= COG_LUT_SIZE) k = COG_LUT_SIZE - 1;
19     return k;
20 }

```

正转采集：

代码块

```

1  void cog_record_positive(float theta_m, float omega_m, float iq_fbk)
2  {
3      uint16_t k = get_cog_index(theta_m);
4
5      float iq_fric = (Bm * omega_m + Cm) / Kt;
6      float iq_res = iq_fbk - iq_fric;
7
8      cog_sum_pos[k] += iq_res;
9      cog_cnt_pos[k] += 1;
10 }

```

反转采集：

代码块

```

1  void cog_record_negative(float theta_m, float omega_m, float iq_fbk)
2  {
3      uint16_t k = get_cog_index(theta_m);
4
5      float iq_fric = (Bm * omega_m - Cm) / Kt;
6      float iq_res = iq_fbk - iq_fric;
7

```

```

8     cog_sum_neg[k] += iq_res;
9     cog_cnt_neg[k] += 1;
10 }

```

生成表：

代码块

```

1  void cog_generate_lut(void)
2  {
3      float mean = 0.0f;
4
5      for (int k = 0; k < COG_LUT_SIZE; k++)
6      {
7          float lut_pos = 0.0f;
8          float lut_neg = 0.0f;
9
10         if (cog_cnt_pos[k] > 0)
11             lut_pos = cog_sum_pos[k] / cog_cnt_pos[k];
12
13         if (cog_cnt_neg[k] > 0)
14             lut_neg = cog_sum_neg[k] / cog_cnt_neg[k];
15
16         cog_lut[k] = 0.5f * (lut_pos + lut_neg);
17         mean += cog_lut[k];
18     }
19
20     mean /= COG_LUT_SIZE;
21
22     for (int k = 0; k < COG_LUT_SIZE; k++)
23     {
24         cog_lut[k] -= mean;
25     }
26
27     // 后面可以再加循环滑动平均
28 }

```

运行时查表

代码块

```

1  float cog_get_comp_current(float theta_m)
2  {
3      float theta = fmodf(theta_m, TWO_PI);
4      if (theta < 0.0f) theta += TWO_PI;
5

```

```

6     float x = theta * ((float)COG_LUT_SIZE / TWO_PI);
7
8     int k0 = (int)x;
9     float alpha = x - (float)k0;
10
11    int k1 = k0 + 1;
12    if (k1 >= COG_LUT_SIZE) k1 = 0;
13
14    return (1.0f - alpha) * cog_lut[k0] + alpha * cog_lut[k1];
15 }

```

速度环中使用：

代码块

```

1  float iq_pi = speed_pi_output;
2
3  float iq_fric = (Bm * omega_m + Cm * sign_func(omega_m)) / Kt;
4
5  float iq_cog = cog_get_comp_current(theta_m);
6
7  float iq_ref = iq_pi + iq_fric + iq_cog;
8
9  iq_ref = limit(iq_ref, -IQ_MAX, IQ_MAX);

```

建议刚开始不要让补偿一下子全开，而是加一个比例系数：

代码块

```

1  float iq_ref = iq_pi + iq_fric + cog_gain * iq_cog;
2  cog_gain = 0.0 ~ 1.0

```

测试时从：cog_gain=0.2慢慢加到cog_gain=1.0

如果加了以后抖动变大，说明表的符号可能反了，可以试：

代码块

```

1  iq_ref = iq_pi + iq_fric - cog_gain * iq_cog;

```

实验五：补偿效果验证

低速匀速测试

分别测试：1 2 5 10 rpm

最终流程

- 1.先确认 FOC 角度和电流采样正确。
- 2.使用你之前辨识得到的 B_m, C_m, K_t 。
先暂时不用 J_m ，因为匀速建表时 $\omega \cdot \approx 0$ 。
- 3.关闭齿槽补偿，低速正转 5 到 10 圈，记录 I_q, θ, ω 。
- 4.关闭齿槽补偿，低速反转 5 到 10 圈，记录 I_q, θ, ω 。
- 5按照位置分桶平均，生成 LUT+ 和 LUT-。
- 6.正反转平均：
- 7.去均值：
- 8.平滑滤波。
- 9.运行时查表插值，叠加到 q 轴电流指令上：
- 10.从小增益开始测试

四、总结

本方法基于电机轴侧动力学模型，将扰动力矩分解为速度相关摩擦项和位置相关齿槽项。首先通过机械参数辨识获得 B_m, C_m ，建立摩擦补偿模型；随后在低速匀速条件下采集 q 轴电流反馈，并扣除摩擦电流分量，得到随电机机械角度变化的齿槽补偿电流。通过正反转采集、多圈平均、去均值和平滑滤波生成齿槽补偿 LUT。运行时根据当前转子机械角度查表插值，将齿槽补偿电流与摩擦补偿电流共同叠加到速度环输出的 q 轴电流指令上，从而降低低速速度波动和周期性转矩脉动。